Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Design Butterworth and Chebyshev filter using bilinear transformation method.**	
Experiment No: 4	**Date:15-10-2026**	**Enrollment No:92400133121**

Aim: Design Butterworth and Chebyshev filter using bilinear transformation method.

Theory:

- The bilinear transformation method is commonly used to design analog filters and then convert them into digital filters.
- This method maps the analog frequency response to the digital frequency response using a bilinear transformation.
- The Butterworth and Chebyshev filters are two commonly used filter types.
- The Butterworth filter has a maximally flat frequency response in the passband, while the Chebyshev filter allows for a sharper transition between the passband and the stopband at the expense of ripples in either the passband or stopband.

Butterworth Filter

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, bilinear, freqz, cheby1

def bwf(filter_order,cutoff_frequency,sampling_frequency):

    analog_b,analog_a = butter(filter_order,cutoff_frequency,analog=True,btype='low') #continuous signal + low pass filter

    print(analog_b)
    print(analog_a)

    digital_b,digital_a = bilinear(analog_b,analog_a,sampling_frequency) #perform bilinear

    return digital_b,digital_a

filter_order = 4
cutoff_frequency = 1000
sampling_frequency = 8000

digital_b,digital_a = bwf(filter_order,cutoff_frequency,sampling_frequency)

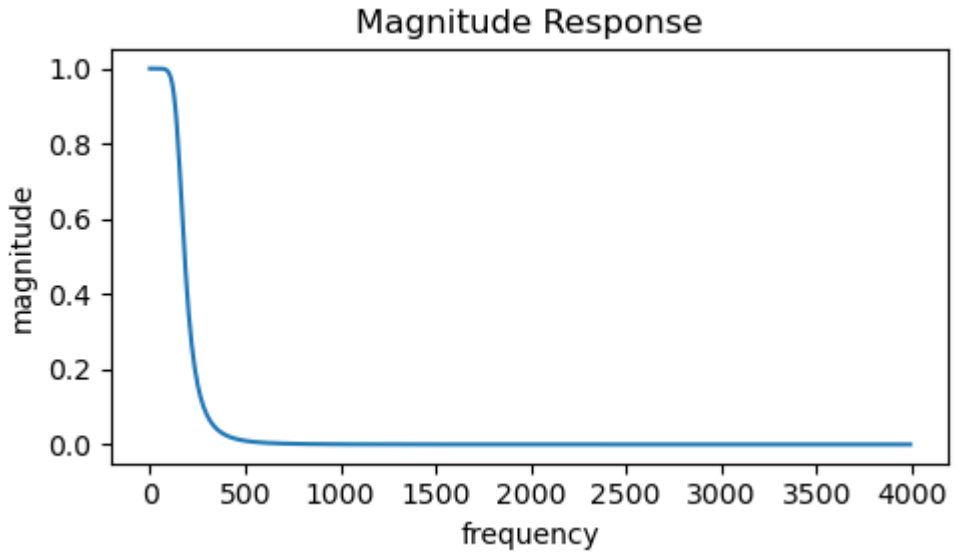
frequency, magnitude_response = freqz(digital_b,digital_a,fs=sampling_frequency)

plt.figure(figsize=(5,3))
plt.plot(frequency,np.abs(magnitude_response))
```

```
plt.title("Magnitude Response")
plt.xlabel("frequency")
plt.ylabel("magnitude")
plt.tight_layout()

plt.show()
```

LOW PASS
ORDER = 4
CUTOFF = 1000

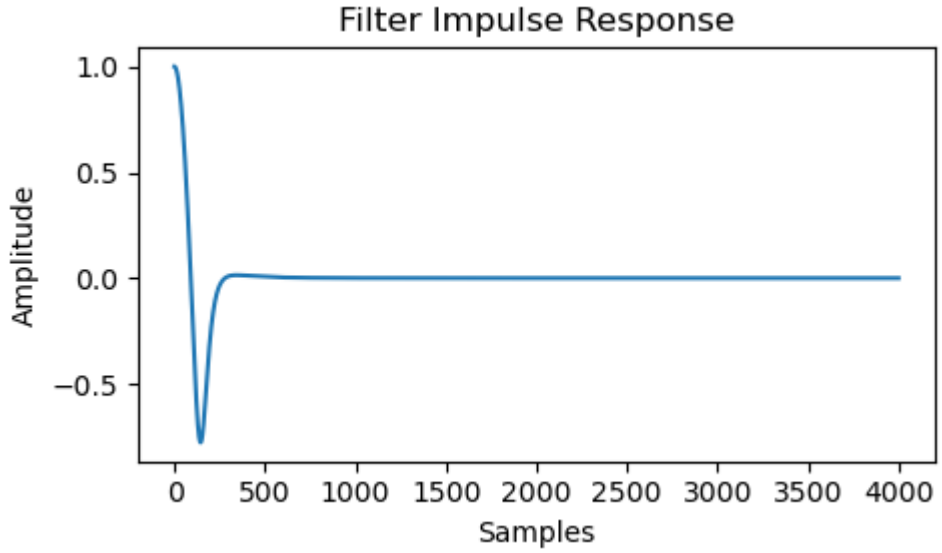



```
frequency,impulse_response = freqz(digital_b,digital_a,fs=sampling_frequency,worN=4000)

plt.figure(figsize=(5,3))
plt.plot(impulse_response)
plt.title("Filter Impulse Response")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.tight_layout()

plt.show()
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Design Butterworth and Chebyshev filter using bilinear transformation method.**	
Experiment No: 4	**Date:15-10-2026**	**Enrollment No:92400133121**



Chebyshev Filter

Code:

```
def cf(filter_order,cutoff_frequency,sampling_frequency,ripple):

    wc = 2*sampling_frequency*np.tan(np.pi*cutoff_frequency/sampling_frequency)
    analog_b,analog_a = cheby1(filter_order,ripple,wc,analog=True,btype='low') #continuous signal +
low pass filter

    print(analog_b)
    print(analog_a)

    digital_b,digital_a = bilinear(analog_b,analog_a,sampling_frequency) #perform bilinear

    return digital_b,digital_a
filter_order = 4
cutoff_frequency = 1000
sampling_frequency = 8000
ripple = 0.5

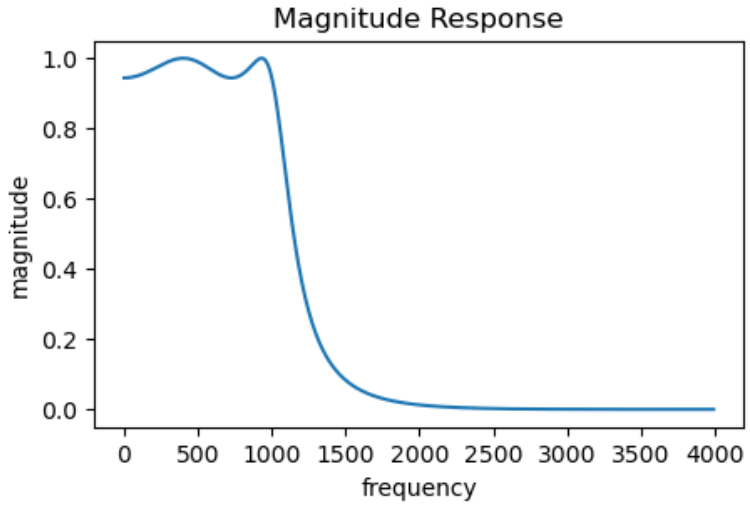
digital_b,digital_a = cf(filter_order,cutoff_frequency,sampling_frequency,ripple)

frequency, magnitude_response = freqz(digital_b,digital_a,fs=sampling_frequency)

plt.figure(figsize=(5,3))
```

```
plt.plot(frequency,np.abs(magnitude_response))
plt.title("Magnitude Response")
plt.xlabel("frequency")
plt.ylabel("magnitude")

plt.show()
```

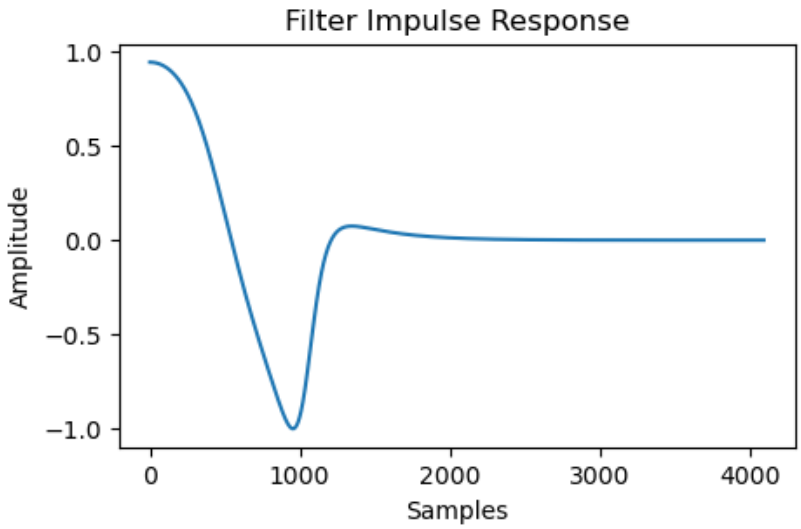



```
frequency,impulse_response = freqz(digital_b,digital_a,fs=sampling_frequency,worN=4096)

plt.figure(figsize=(5,3))
plt.plot(impulse_response)
plt.title("Filter Impulse Response")
plt.xlabel("Samples")
plt.ylabel("Amplitude")

plt.show()
```

Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**		
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Design Butterworth and Chebyshev filter using bilinear transformation method.**	
Experiment No: 4	**Date:15-10-2026**	**Enrollment No:92400133121**



Conclusion

- Butterworth and Chebyshev low-pass digital filters were designed using the bilinear transformation method.
- The analog filter coefficients were calculated and converted into digital filter coefficients using bilinear transformation.
- The magnitude response and impulse response of both filters were plotted and observed.
- The Butterworth filter showed a smooth, maximally flat passband response.
- The Chebyshev filter showed a sharper transition from the passband to the stopband, with a small ripple in the passband.

ADDITIONAL TASKS

#TASK1 - Generate a discrete sine wave of 1 KHz,2KHz, 3 KHz, 4 KHz. Plot all these

```
sampling_frequency = 16000
t = np.arange(0, 1, 1/sampling_frequency)

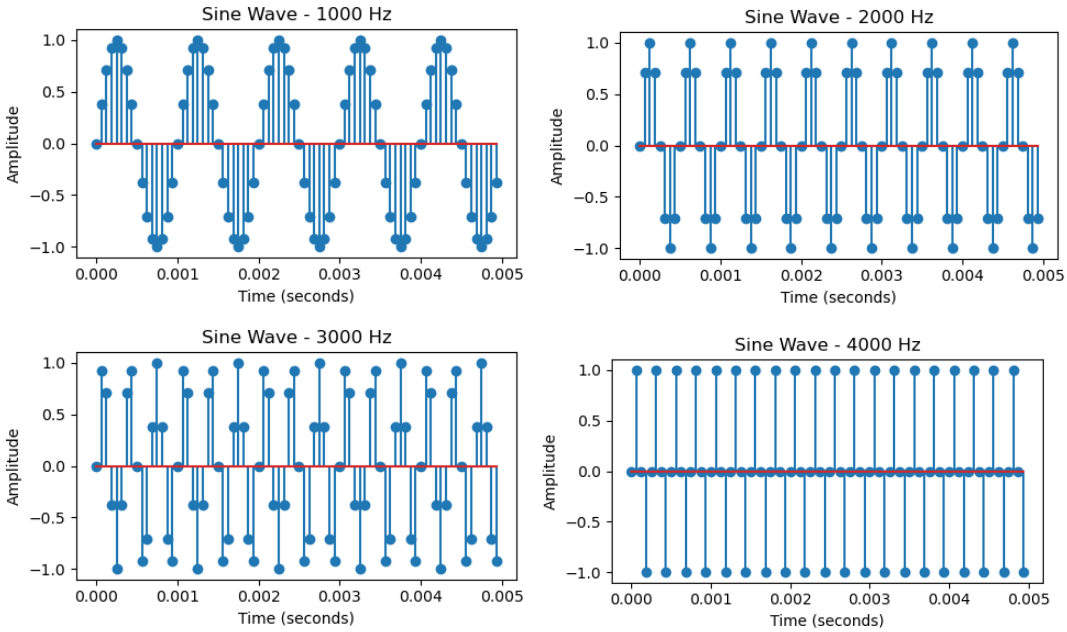
x1 = np.sin(2*np.pi*1000*t)
x2 = np.sin(2*np.pi*2000*t)
x3 = np.sin(2*np.pi*3000*t)
x4 = np.sin(2*np.pi*4000*t)

signals = [x1, x2, x3, x4]
frequencies = [1000, 2000, 3000, 4000]

for i in range(4):
```

```
plt.figure(figsize=(5, 3))
samples = int(0.005 * sampling_frequency)
plt.stem(t[:samples], signals[i][:samples])
plt.title(f"Sine Wave - {frequencies[i]} Hz")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()
```

Output:



#TASK2 - Add all these four signal and plot it in time as well as frequency domain

combined_signal = x1 + x2 + x3 + x4

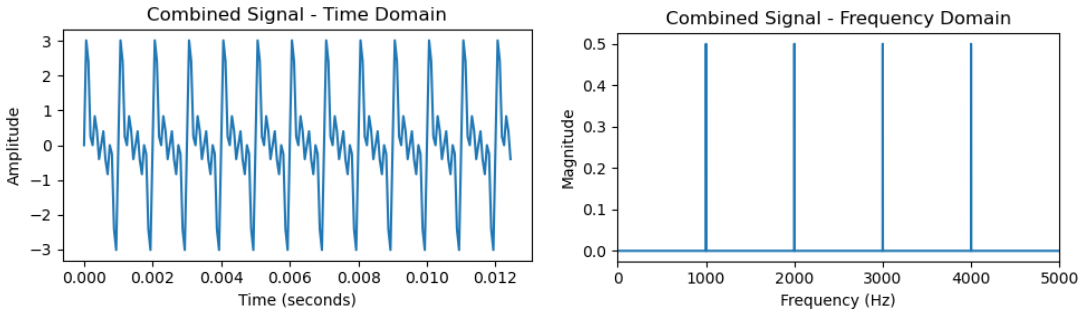
```
# Time domain
plt.figure(figsize=(5, 3))
plt.plot(t[:200], combined_signal[:200])
plt.title("Combined Signal - Time Domain")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()

# Frequency domain
```

```
N = len(combined_signal)
frequency = np.fft.rfftfreq(N, 1/sampling_frequency)
magnitude = np.abs(np.fft.rfft(combined_signal))/N

plt.figure(figsize=(5, 3))
plt.plot(frequency, magnitude)
plt.title("Combined Signal - Frequency Domain")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.xlim(0, 5000)
plt.tight_layout()
plt.show()
```

Output:



#TASK3 - Pass through butterworth digital filter having cutff frequency 2 KHz

```
filter_order = 4
cutoff_frequency = 2000

digital_b, digital_a = bwf(
   filter_order, cutoff_frequency, sampling_frequency
)

butterworth_output = np.zeros_like(combined_signal)

# Apply the filter using the difference equation
from scipy.signal import lfilter
butterworth_output = lfilter(
   digital_b, digital_a, combined_signal
)

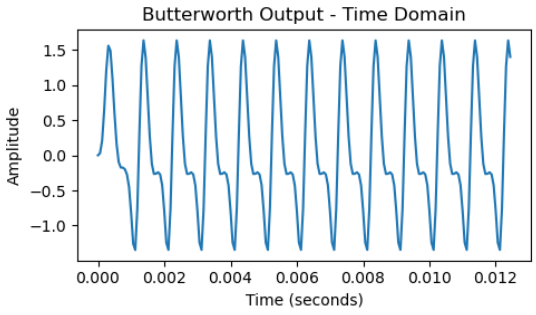
# Time domain
```

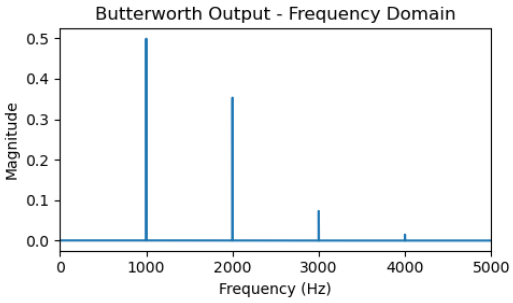
```
plt.figure(figsize=(5, 3))
plt.plot(t[:200], butterworth_output[:200])
plt.title("Butterworth Output - Time Domain")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()

# Frequency domain
frequency = np.fft.rfftfreq(N, 1/sampling_frequency)
magnitude = np.abs(np.fft.rfft(butterworth_output))/N

plt.figure(figsize=(5, 3))
plt.plot(frequency, magnitude)
plt.title("Butterworth Output - Frequency Domain")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.xlim(0, 5000)
plt.tight_layout()
plt.show()
```

Output:





#TASK-4 : Pass through chebyschev digital filter having cutoff frequency 3 KHz and plot the output plot in time as well as frequency domain

```
filter_order = 4
cutoff_frequency = 3000
ripple = 0.5

digital_b, digital_a = cf(
    filter_order, cutoff_frequency,
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Design Butterworth and Chebyshev filter using bilinear transformation method.**	
Experiment No: 4	**Date:15-10-2026**	**Enrollment No:92400133121**

```
    sampling_frequency, ripple
)

chebyshev_output = lfilter(
    digital_b, digital_a, combined_signal
)

# Time domain
plt.figure(figsize=(5, 3))
plt.plot(t[:200], chebyshev_output[:200])
plt.title("Chebyshev Output - Time Domain")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()

# Frequency domain
frequency = np.fft.rfftfreq(N, 1/sampling_frequency)
magnitude = np.abs(np.fft.rfft(chebyshev_output))/N

plt.figure(figsize=(5, 3))
plt.plot(frequency, magnitude)
plt.title("Chebyshev Output - Frequency Domain")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.xlim(0, 5000)
plt.tight_layout()
plt.show()
```

Output:

